FINAL EXPERIMENTAL REPORT

Pruning vs XNOR-Net

Reduced-load reproduction for environmental sound classification

ESC-10 subset of ESC-50 • Five-fold assessment • 12 November 2026

Executive summary

This report documents a computationally reduced reproduction of the audio experiments in Mohaimenuzzaman, Bergmeir and Meyer (2022), “Pruning vs XNOR-Net.” Using the authors’ network implementations and a prepared ESC-50 dataset, the project trained Mini-ACDNet and XMini-ACDNet for 200 epochs per fold, then evaluated one-shot structured channel pruning and CPU INT8 post-training quantization using existing checkpoints.

The intent was to verify a functioning end-to-end comparison under a laptop GPU budget—not to recreate the original paper’s complete training matrix or resource-equivalent QMicro-vs-XMini benchmark.

Key findings

Result	Mini / pruned INT8	XNOR counterpart
200-epoch five-fold validation	Mini FP32: 91.25%	XMini: 69.25%
Lightweight five-fold evaluation	Pruned + INT8: 72.50%	XMini: 58.00%
Model compression	9.83% fewer FP32 parameters	No bit-packed storage measured

Scope and interpretation

- Training-based results use all 80 held-out ESC-10 clips per fold and 10-crop evaluation; checkpoint accuracy is the best observed across evaluations.
- Compression-based results evaluate 40 balanced clips per fold with three crops each; these must not be pooled with the training results.
- The 5% TFEB channel-pruning setting removed 102,875 of 1,046,396 FP32 parameters. Eager INT8 conversion and inference succeeded on oneDNN.
- The study supports functionality and directional accuracy observations, not claims about equal-memory model superiority or MCU runtime speed.

Report basis: console outputs and local-script source provided in this project, plus the uploaded 2022 IEEE Access article. Model files and result CSVs reside on the user’s Windows machine and were not uploaded for independent re-evaluation.

1 Experimental methodology

1.1 Dataset, folds and input

The experiment used the ESC-10 subset of the ESC-50 environmental sound dataset, stored as wav20.npz. ESC-50 provides 2,000 recordings across five predefined folds; the selected 10 classes contain 400 recordings total. For each held-out fold, 320 recordings were available for training and 80 for validation, without training-record overlap between folds.

Input was raw mono audio at 20 kHz, represented by 30,225 waveform samples per crop (~1.51 seconds). The project training runner generated augmented Between-Class (BC) mixtures online, with randomized scaling, cropping, mixing and gain. Validation averaged predictions from 10 crops per recording; the later lightweight comparison used three crops.

1.2 Runtime and training

Component	Configuration
Environment	Windows / VS Code; Python 3.10.11; PyTorch 2.9.1+cu128
Compute	NVIDIA RTX 4060 Laptop GPU (FP32 and XMini); CPU/oneDNN (INT8)
Models	Mini-ACDNet and XMini-ACDNet; existing repository architectures
Training	200 epochs per fold; batch size 64; five fold-specific models
Optimization	Mini: SGD / Nesterov with step schedule; XMini: Adam / cosine schedule
Evaluation	Every 10 epochs; best checkpoint chosen on evaluated fold
Source checkpoint	Fold 1 from runner v1; Folds 2–5 from v2 (checkpoint resume added)

1.3 Lightweight compression protocol

For each fold, the trained Mini checkpoint was evaluated alongside XMini. A one-shot layer-wise L2 filter norm ranked channels in the 10 TFEB convolution layers; a stated fraction of channels was physically removed, with connected BatchNorm and downstream layers rewired. The pruned network was evaluated before optional short fine-tuning.

For INT8 post-training quantization, supported Conv–BatchNorm–ReLU blocks were fused, the pruned model was calibrated on 12 training recordings, and PyTorch eager quantization converted supported convolution operations for CPU inference using oneDNN. The quantized model was traced, exported, reloaded and checked for output consistency.

1.4 Controlled differences from the paper

The published study trained the full range of ESC-10/20/30/40/50 tasks for 2,000 epochs and used an iterative hybrid Taylor-scored pruning/fine-tuning procedure for deriving Micro models, followed by 8-bit quantization. This reproduction instead uses only ESC-10, 200 training epochs, and one-shot L2 pruning on pre-trained Mini models; it does not create a paper-equivalent QMicro-ACDNet.

2 Full ESC-10 five-fold training evaluation

Table 1. Best held-out-fold validation accuracies (%) after 200 training epochs; 80 validation clips and 10 crops per fold.

Evaluation fold	Mini FP32 (%)	XMini (%)	Difference (pp)
Fold 1	87.50	63.75	+23.75
Fold 2	92.50	72.50	+20.00
Fold 3	88.75	66.25	+22.50
Fold 4	96.25	70.00	+26.25
Fold 5	91.25	73.75	+17.50
Five-fold mean	**91.25**	**69.25**	**+22.00**

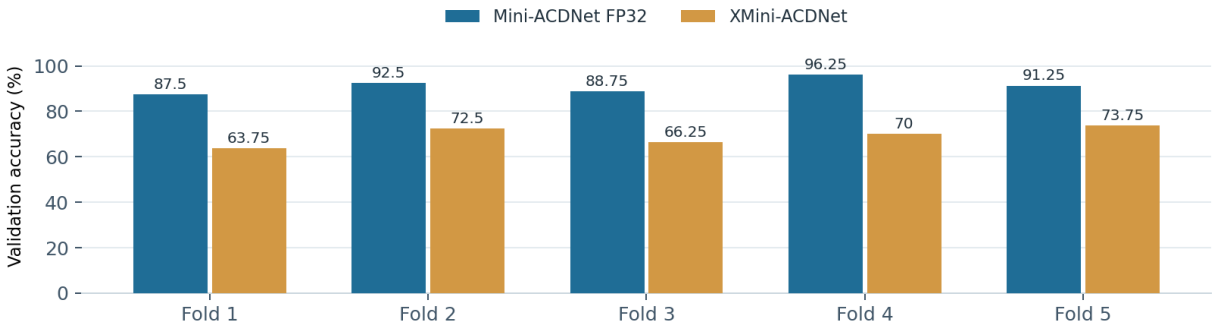


Figure 1. Accuracy for each held-out fold under the longer 200-epoch protocol. The figure uses the best validation checkpoint for each fold.

2.1 Interpretation

Mini-ACDNet outperformed XMini-ACDNet on every fold. The mean difference was 22.00 percentage points; the best-checkpoint mean was 91.25% for Mini (sample standard deviation 3.42 pp) versus 69.25% for XMini (4.20 pp). These five folds represent different held-out recordings; their variation should not be treated as five independent random training repetitions.

2.2 Published ESC-10 reference points

Model	Original paper (%)	This project, 200 epochs (%)	Gap (pp)
Mini-ACDNet	96.75	91.25	−5.50
XMini-ACDNet	82.25	69.25	−13.00
Micro-ACDNet	96.25	Not trained	—
QMicro-ACDNet	92.75	Not trained	—

Source: paper Tables 3 and 6 (IEEE Access, 2022, pp. 6701–6702). Differences are descriptive only: the paper trained longer, and best-fold checkpoint selection introduces possible optimism in these project results.

3 Lightweight five-fold compression evaluation

The following tests reused the 200-epoch checkpoints. Each fold was evaluated on a seeded balanced subset of 40 held-out recordings (four recordings per class) with three prediction crops per recording. Therefore, these percentages are not interchangeable with Section 2 values.

Table 2. Lightweight five-fold accuracy (%) for the final 5%-channel setting with no fine-tuning.

Fold	Mini FP32	XMini	Pruned FP32	Pruned INT8
Fold 1	80.00	55.00	62.50	60.00
Fold 2	80.00	57.50	70.00	70.00
Fold 3	90.00	55.00	82.50	77.50
Fold 4	90.00	60.00	90.00	87.50
Fold 5	82.50	62.50	75.00	67.50
Mean	**84.50**	**58.00**	**76.00**	**72.50**

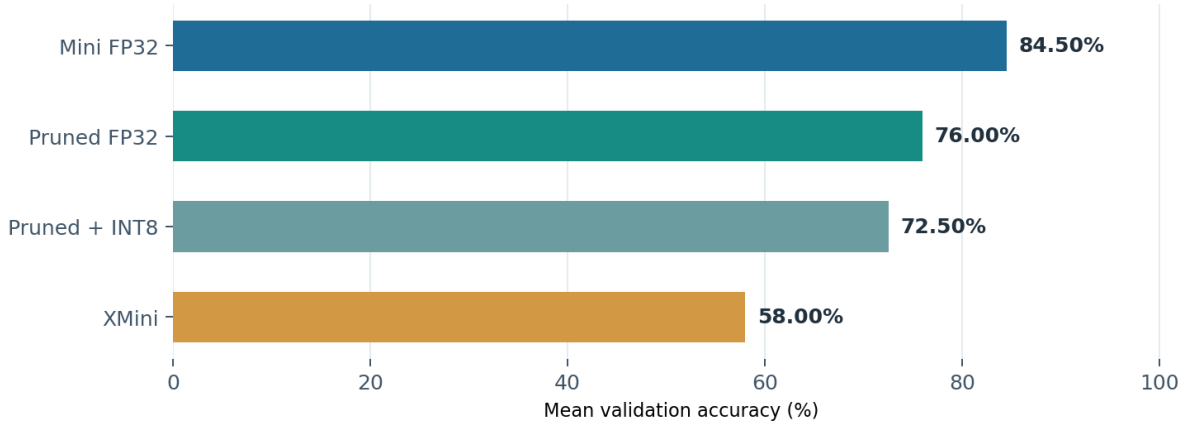


Figure 2. Mean accuracies from Table 2. The INT8 variant uses one-shot 5% TFEB pruning, no fine-tuning, and 12 calibration clips per fold.

3.1 Accuracy changes

The 5% pruning setting lowered mean accuracy from 84.50% to 76.00% (−8.50 pp). Direct INT8 quantization reduced it a further 3.50 pp to 72.50%. The quantized model exceeded XMini by 14.50 pp under this reduced evaluation, but the two were not matched for deployed memory or binary compute cost.

3.2 Pruning intensity and brief fine-tuning

Setting	Pruned FP32	After short tune	After tune + INT8
20% TFEB pruning; 1 epoch × 2 steps	21.00%	34.00%	34.00%
5% TFEB pruning; 3 epochs × 10 steps	76.00%	69.00%	70.00%
5% TFEB pruning; 0 fine-tune epochs	76.00%	76.00%*	72.50%*

** "After tune" is an unchanged pruned model when zero epochs were requested; the INT8 row has no fine-tuning. The short tuning did not consistently improve validation accuracy.*

4 Compression and paper comparison

4.1 Structural parameter reduction

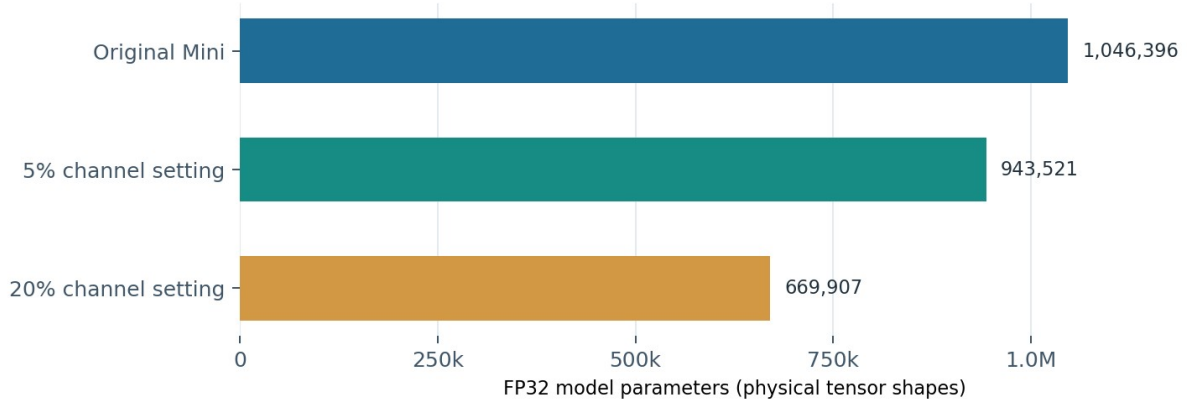


Figure 3. Physical FP32 parameter counts after one-shot pruning of the TFEB portion of the model.

Configuration	Parameter count	Removed	Reduction
Mini FP32 (loaded checkpoint)	1,046,396	—	—
5% channel setting	943,521	102,875	9.83%
20% channel setting	669,907	376,489	35.98%

The pruning fraction describes the fraction of output channels removed within each targeted TFEB layer—not the whole-model parameter reduction. Pruning changes the physical tensor dimensions. A parameter count is not a direct measure of quantized deployed file size or runtime memory.

4.2 Resource comparison with the original paper

Published architecture	Accuracy on ESC-10	Paper-reported model size
Mini-ACDNet (FP32)	96.75%	4,096 KB
XMini-ACDNet (XNOR)	82.25%	128.5 KB
QMicro-ACDNet (8-bit)	92.75%	133.12 KB

Sources: paper Tables 2, 3, 5 and 6, pp. 6701–6702. The paper's model-size values are theoretical/estimated storage figures for its own architectures, not measurements from this project.

The original comparison was designed around approximately equal XMini and QMicro storage budgets. This project quantized a lightly pruned Mini model instead of constructing QMicro, so its 72.50% vs 58.00% accuracy comparison cannot demonstrate the paper's matched-memory claim. The PyTorch XMini also did not use a deployed, bit-packed XNOR kernel; compute/memory speedups on microcontrollers were not measured.

4.3 Functionality verification

Separate smoke tests confirmed structural removal with working forward/backward propagation, oneDNN-based INT8 conversion of 12 convolution layers, CPU INT8 inference, traced export and reload. The later five-fold lightweight script also exported and reloaded INT8 models and checked output agreement. A oneDNN warning about CPU vector instruction support was non-fatal and should be considered when interpreting accuracy or performance on other CPUs.

5 Limitations, conclusions and reproducibility

5.1 Principal limitations

- Not a full methodological replication: 200 rather than 2,000 epochs, ESC-10 only, and no ESC-20/30/40/50, AclNet or UrbanSound8k results.
- Different compression algorithm: one-shot, local L2-ranked TFEB channel pruning, not global iterative Taylor-scored pruning with repeated recovery training.
- No matched-memory QMicro vs XMini comparison: parameter counts were checked, but deployed flash/RAM, native bit packing, FLOPs and latency were not benchmarked.
- Optimistic checkpoint selection: the same held-out fold was used to choose the best model checkpoint and to report accuracy.
- Reduced evaluation uncertainty: only four validation recordings per class (40 per fold) and three crops in compression tests, versus eight per class and ten crops for the longer evaluation.
- Limited calibration and tuning: 12 calibration clips and very short fine-tuning may not optimize INT8 or recover from pruning; results should not be generalized to deployed MCU hardware.
- Evidence base: results were transcribed from user-provided console logs and script source; source CSVs, model binaries and end-device profiling data were not independently supplied for auditing.

5.2 Conclusions

The reduced-load reproduction accomplished its engineering objective: the two ACDNet variants trained across the five predefined ESC-10 folds; physical channel pruning preserved functional forward and backward computation; quantized convolution inference and export worked on CPU. On the full 200-epoch evaluation, Mini outperformed XMini (91.25% vs 69.25%). On the smaller matched-evaluation subset, the 5%-pruned INT8 variant achieved 72.50% and XMini achieved 58.00%.

The pruning sweep highlights a practical constraint: aggressive one-shot pruning degraded accuracy sharply, while gentle pruning retained more signal. Brief fine-tuning was not reliably beneficial. These findings provide a reproducible demonstration, but they do not reproduce the paper's broader model compression ranking under equal hardware and memory constraints.

5.3 Reproducibility record

Local project artifact	Role
esc50_repro_runner_v2.py	200-epoch training and ten-crop validation
esc50_lite_reproduction.py	Five-fold L2 pruning and INT8 comparison
experiments/esc50/mini10_*; xmini10_*	Best checkpoints and training history CSVs
experiments/esc50/lite_prune5_no_finetune/comparison.csv	Final reduced evaluation result rows
experiments/esc50/lite_prune5_fivefold/comparison.csv	Three-epoch tuning comparison
experiments/esc50/lite_fivefold/comparison.csv	Aggressive pruning comparison

References

[1] M. Mohaimenuzzaman, C. Bergmeir, and B. Meyer, "Pruning vs XNOR-Net: A Comprehensive Study of Deep Learning for Audio Classification on Edge-Devices," IEEE Access, vol. 10, pp. 6696–6707, 2022. DOI: 10.1109/ACCESS.2022.3140807.

[2] Official source repository: https://github.com/mohaimenz/pruning-vs-xnor. Local evaluation utilities and console outputs described in this report are project-specific adaptations.